Práctica 1

Generated by Doxygen 1.9.8

1 Test List 1

2 Data Structure Index 3

2.1 Data Structures . . . 3

3 File Index 5

3.1 File List . . . 5

4 Data Structure Documentation 7

4.1 Arg_hilos Struct Reference . . . 7

4.1.1 Detailed Description . . . 7

4.1.2 Field Documentation . . . 7

4.1.2.1 found_value . . . 7

4.1.2.2 max . . . 7

4.1.2.3 min . . . 8

4.1.2.4 target . . . 8

4.2 Logger_args Struct Reference . . . 8

4.2.1 Detailed Description . . . 8

4.2.2 Field Documentation . . . 8

4.2.2.1 id . . . 8

4.2.2.2 pos_votes . . . 9

4.2.2.3 solution . . . 9

4.2.2.4 target . . . 9

4.2.2.5 validated . . . 9

4.2.2.6 votes . . . 9

4.2.2.7 wallets . . . 9

4.2.2.8 winner . . . 9

4.3 Miner_data Struct Reference . . . 10

4.3.1 Detailed Description . . . 10

4.3.2 Field Documentation . . . 10

4.3.2.1 n_threads . . . 10

4.3.2.2 time . . . 10

4.4 Miner_Mutexes Struct Reference . . . 10

4.4.1 Detailed Description . . . 11

4.4.2 Field Documentation . . . 11

4.4.2.1 pid . . . 11

4.4.2.2 tgt . . . 11

4.4.2.3 vot . . . 11

4.4.2.4 win . . . 11

5 File Documentation 13

5.1 files.c File Reference . . . 13

5.1.1 Detailed Description . . . 14

5.1.2 Function Documentation . . . 14

5.1.2.1 close_pipes() . . . 14
5.1.2.2 count_votes() . . . 15
5.1.2.3 get_active_pids_unlocked() . . . 15
5.1.2.4 initialize_mutexes() . . . 15
5.1.2.5 open_pipes() . . . 16
5.1.2.6 read_target_unlocked() . . . 16
5.1.2.7 write_pid_unlocked() . . . 16
5.1.2.8 write_target_unlocked() . . . 17
5.1.2.9 write_vote() . . . 17
5.2 files.h File Reference . . . 17
5.2.1 Detailed Description . . . 19
5.2.2 Macro Definition Documentation . . . 19
5.2.2.1 PID_FILE . . . 19
5.2.2.2 PID_MUTEX . . . 20
5.2.2.3 READ . . . 20
5.2.2.4 TARGET_FILE . . . 20
5.2.2.5 TARGET_MUTEX . . . 20
5.2.2.6 VOTES_FILE . . . 20
5.2.2.7 VOTES_MUTEX . . . 20
5.2.2.8 WINNER_MUTEX . . . 20
5.2.2.9 WRITE . . . 20
5.2.3 Function Documentation . . . 20
5.2.3.1 close_pipes() . . . 20
5.2.3.2 count_votes() . . . 21
5.2.3.3 get_active_pids_unlocked() . . . 21
5.2.3.4 initialize_mutexes() . . . 22
5.2.3.5 open_pipes() . . . 22
5.2.3.6 read_target_unlocked() . . . 22
5.2.3.7 write_pid_unlocked() . . . 22
5.2.3.8 write_target_unlocked() . . . 23
5.2.3.9 write_vote() . . . 23
5.3 files.h . . . 23
5.4 logger.c File Reference . . . 24
5.4.1 Detailed Description . . . 25
5.4.2 Function Documentation . . . 25
5.4.2.1 logger() . . . 25
5.5 logger.h File Reference . . . 26
5.5.1 Detailed Description . . . 27
5.5.2 Function Documentation . . . 27
5.5.2.1 logger() . . . 27
5.6 logger.h . . . 27
5.7 miner.c File Reference . . . 28

5.7.1 Detailed Description . . . 29
5.7.2 Macro Definition Documentation . . . 30
5.7.2.1 TARGET_INIT . . . 30
5.7.3 Function Documentation . . . 30
5.7.3.1 calcular_solucion() . . . 30
5.7.3.2 comunicar_logger() . . . 30
5.7.3.3 exit_network() . . . 31
5.7.3.4 get_time_str() . . . 31
5.7.3.5 handler() . . . 31
5.7.3.6 miner_set_alarm() . . . 31
5.7.3.7 minero() . . . 32
5.7.3.8 pow_seek() . . . 32
5.7.3.9 wait_more_miners() . . . 32
5.7.3.10 wait_votes() . . . 33
5.7.4 Variable Documentation . . . 33
5.7.4.1 start_mining . . . 33
5.7.4.2 start_voting . . . 33
5.7.4.3 timeout . . . 33
5.8 unit_tests/miner_test/miner.c File Reference . . . 34
5.8.1 Detailed Description . . . 35
5.8.2 Macro Definition Documentation . . . 35
5.8.2.1 TARGET_INIT . . . 35
5.8.3 Function Documentation . . . 36
5.8.3.1 calcular_solucion() . . . 36
5.8.3.2 comunicar_logger() . . . 36
5.8.3.3 exit_network() . . . 36
5.8.3.4 get_time_str() . . . 37
5.8.3.5 handler() . . . 37
5.8.3.6 miner_set_alarm() . . . 37
5.8.3.7 minero() . . . 37
5.8.3.8 pow_seek() . . . 38
5.8.3.9 wait_more_miners() . . . 38
5.8.3.10 wait_votes() . . . 38
5.8.4 Variable Documentation . . . 39
5.8.4.1 start_mining . . . 39
5.8.4.2 start_voting . . . 39
5.8.4.3 timeout . . . 39
5.9 miner.h File Reference . . . 39
5.9.1 Detailed Description . . . 40
5.9.2 Function Documentation . . . 41
5.9.2.1 minero() . . . 41
5.10 miner.h . . . 41

5.11 unit_tests/miner_test/miner.h File Reference . . . 41
5.11.1 Detailed Description . . . 43
5.11.2 Function Documentation . . . 43
5.11.2.1 calcular_solucion() . . . 43
5.11.2.2 minero() . . . 44
5.11.2.3 wait_more_miners() . . . 44
5.11.2.4 wait_votes() . . . 44
5.12 miner.h . . . 45
5.13 mrush.c File Reference . . . 45
5.13.1 Detailed Description . . . 46
5.13.2 Function Documentation . . . 46
5.13.2.1 str_to_u64() . . . 46
5.14 pow.h File Reference . . . 47
5.14.1 Detailed Description . . . 48
5.14.2 Macro Definition Documentation . . . 48
5.14.2.1 POW_LIMIT . . . 48
5.14.3 Function Documentation . . . 48
5.14.3.1 pow_hash() . . . 48
5.15 pow.h . . . 49
5.16 types.h File Reference . . . 49
5.16.1 Detailed Description . . . 50
5.16.2 Macro Definition Documentation . . . 51
5.16.2.1 die . . . 51
5.16.2.2 die_msg . . . 52
5.16.2.3 ERR . . . 52
5.16.2.4 FOUND . . . 52
5.16.2.5 MAX_MINERS . . . 52
5.16.2.6 MAX_TRIES . . . 52
5.16.2.7 MIN . . . 52
5.16.2.8 MIN_MINERS . . . 53
5.16.2.9 OK . . . 53
5.16.3 Typedef Documentation . . . 53
5.16.3.1 f32 . . . 53
5.16.3.2 f64 . . . 53
5.16.3.3 i32 . . . 53
5.16.3.4 i64 . . . 53
5.16.3.5 u32 . . . 54
5.16.3.6 u64 . . . 54
5.17 types.h . . . 54
5.18 unit_tests/files_test/files_test.c File Reference . . . 54
5.18.1 Detailed Description . . . 55
5.18.2 Function Documentation . . . 56

5.18.2.1 test1_open_pipes() . . . 56
5.18.2.2 test2_close_pipes() . . . 56
5.18.2.3 test3_mutex_init() . . . 56
5.18.2.4 test4_pids_io() . . . 56
5.18.2.5 test5_voting_system() . . . 56
5.19 unit_tests/files_test/files_test.h File Reference . . . 57
5.19.1 Detailed Description . . . 57
5.19.2 Function Documentation . . . 58
5.19.2.1 test1_open_pipes() . . . 58
5.19.2.2 test2_close_pipes() . . . 58
5.19.2.3 test3_mutex_init() . . . 58
5.19.2.4 test4_pids_io() . . . 58
5.19.2.5 test5_voting_system() . . . 58
5.20 files_test.h . . . 59
5.21 logger_test.h . . . 59
5.22 unit_tests/miner_test/miner_test.c File Reference . . . 59
5.22.1 Detailed Description . . . 60
5.22.2 Function Documentation . . . 60
5.22.2.1 test1_calcular_solucion() . . . 60
5.22.2.2 test1_open_pipes() . . . 60
5.22.2.3 test1_wait_more_miners_success() . . . 61
5.22.2.4 test1_wait_votes_success() . . . 61
5.22.2.5 test2_wait_more_miners_timeout() . . . 61
5.22.2.6 test2_wait_votes_timeout() . . . 61
5.22.3 Variable Documentation . . . 61
5.22.3.1 start_voting . . . 61
5.22.3.2 timeout . . . 62
5.23 unit_tests/miner_test/miner_test.h File Reference . . . 62
5.23.1 Detailed Description . . . 62
5.23.2 Function Documentation . . . 63
5.23.2.1 test1_calcular_solucion() . . . 63
5.23.2.2 test1_wait_more_miners_success() . . . 63
5.23.2.3 test1_wait_votes_success() . . . 63
5.23.2.4 test2_wait_more_miners_timeout() . . . 63
5.23.2.5 test2_wait_votes_timeout() . . . 63
5.24 miner_test.h . . . 64
5.25 unit_tests/pow_test/pow_test.c File Reference . . . 64
5.25.1 Detailed Description . . . 65
5.25.2 Function Documentation . . . 65
5.25.2.1 main() . . . 65
5.25.2.2 test1_pow() . . . 65
5.26 unit_tests/pow_test/pow_test.h File Reference . . . 66

5.26.1 Detailed Description 66
5.26.2 Function Documentation 67
5.26.2.1 test1_pow() 67
5.27 pow_test.h 67
5.28 test.h 67
5.29 test.h 68
5.30 test.h 68
5.31 test.h 68
5.32 test.h 69

Index **71**

Chapter 1

Test List

Global test1_calcular_solucion ()

Test calcular_solucion

Global test1_open_pipes ()

Test open_pipes

Global test1_pow ()

Test pow function

Global test1_wait_more_miners_success ()

Test wait_more_miners (Success)

Global test1_wait_votes_success ()

Test wait_votes (Success)

Global test2_close_pipes ()

Test close_pipes

Global test2_wait_more_miners_timeout ()

Test wait_more_miners (fracaso timeout)

Global test2_wait_votes_timeout ()

Test wait_votes (fracaso timeout)

Global test3_mutex_init ()

Test inicialización de semáforos

Global test4_pids_io ()

Test lectura y escritura de PIDs

Global test5_voting_system ()

Test del sistema de votaciones

Chapter 2

Data Structure Index

2.1 Data Structures

Here are the data structures with brief descriptions:

Arg_hilos
Estructura para manejo de argumentos de hilos Servira para guardar valores para busqueda de hash a traves de POW 7

Logger_args
Argumentos y datos del bloque minado Empleamos esta estructura para manejar los argumentos del registrador Son los argumentos que almacenara el registrador en el fichero 8

Miner_data
Estructura para manejo de argumentos de Minero Servira para guardar los datos relevantes para el minero 10

Miner_Mutexes
Define el struct de mutexes que se usan en el proyecto 10

Chapter 3

File Index

3.1 File List

Here is a list of all documented files with brief descriptions:

files.c
Implementacion de manejo de archivos 13
files.h
Interfaz de manejo de archivos 17
logger.c
Implementación de registrador 24
logger.h
Interfaz de registrador 26
miner.c
Implementación de minero 28
miner.h
Interfaz de mineros 39
mrush.c
Punto de entrada del programa 45
pow.h
Computation of the POW 47
types.h
Definición de tipos de datos globales 49
unit_tests/test.h 69
unit_tests/files_test/files_test.c
Testea el modulo de archivos 54
unit_tests/files_test/files_test.h
Declara los tests para el modulo de archivos y semaforos (files.c) 57
unit_tests/files_test/test.h 67
unit_tests/logger_test/logger_test.h 59
unit_tests/logger_test/test.h 68
unit_tests/miner_test/miner.c
Implementación de minero 34
unit_tests/miner_test/miner.h
Interfaz de mineros 41
unit_tests/miner_test/miner_test.c
Testea el modulo Miner 59
unit_tests/miner_test/miner_test.h
Declara los tests para el modulo miner 62
unit_tests/miner_test/test.h 68

unit_tests/pow_test/pow_test.c
Testea el modulo pow 64
unit_tests/pow_test/pow_test.h
Declara los tests para la funcion POW 66
unit_tests/pow_test/test.h 68

Chapter 4

Data Structure Documentation

4.1 Arg_hilos Struct Reference

Estructura para manejo de argumentos de hilos Servira para guardar valores para busqueda de hash a traves de POW.

```
#include <miner.h>
```

Data Fields

- u64 target
- u64 min
- u64 max
- volatile int * found_value

4.1.1 Detailed Description

Estructura para manejo de argumentos de hilos Servira para guardar valores para busqueda de hash a traves de POW.

4.1.2 Field Documentation

4.1.2.1 found_value

```
volatile int * Arg_hilos::found_value
```

Booleano de hallazgo de la solucion

4.1.2.2 max

```
u64 Arg_hilos::max
```

Valor maximo de busqueda

4.1.2.3 min

```
u64 Arg_hilos::min
```

Valor minimo de busqueda

4.1.2.4 target

```
u64 Arg_hilos::target
```

Objetivo de busqueda de funcion hash

The documentation for this struct was generated from the following files:

- miner.h
- unit_tests/miner_test/miner.h

4.2 Logger_args Struct Reference

Argumentos y datos del bloque minado Empleamos esta estructura para manejar los argumentos del registrador Son los argumentos que almacenara el registrador en el fichero.

```
#include <logger.h>
```

Data Fields

- u64 id
- u64 target
- u64 solution
- u64 pos_votes
- u64 votes
- u64 wallets
- pid_t winner
- i32 validated

4.2.1 Detailed Description

Argumentos y datos del bloque minado Empleamos esta estructura para manejar los argumentos del registrador Son los argumentos que almacenara el registrador en el fichero.

4.2.2 Field Documentation

4.2.2.1 id

```
u64 Logger_args::id
```

Numero de ronda

4.2.2.2 pos_votes

```
u64 Logger_args::pos_votes
```

Numero de votos a favor

4.2.2.3 solution

```
u64 Logger_args::solution
```

Valor que resuelve el POW

4.2.2.4 target

```
u64 Logger_args::target
```

Objetivo de la ronda

4.2.2.5 validated

```
i32 Logger_args::validated
```

Booleano que representa validez de la solucion

4.2.2.6 votes

```
u64 Logger_args::votes
```

Numero de votos totales

4.2.2.7 wallets

```
u64 Logger_args::wallets
```

Id padre:Rondas ganadas

4.2.2.8 winner

```
pid_t Logger_args::winner
```

ID Proceso ganador

The documentation for this struct was generated from the following file:

- logger.h

4.3 Miner_data Struct Reference

Estructura para manejo de argumentos de Minero Servira para guardar los datos relevantes para el minero.

```
#include <miner.h>
```

Data Fields

- u64 time
- u64 n_threads

4.3.1 Detailed Description

Estructura para manejo de argumentos de Minero Servira para guardar los datos relevantes para el minero.

4.3.2 Field Documentation

4.3.2.1 n_threads

```
u64 Miner_data::n_threads
```

Numero de hilos en los que separar carga de trabajo

4.3.2.2 time

```
u64 Miner_data::time
```

Tiempo en segundos de ejecucion del minero

The documentation for this struct was generated from the following files:

- miner.h
- unit_tests/miner_test/miner.h

4.4 Miner_Mutexes Struct Reference

Define el struct de mutexes que se usan en el proyecto.

```
#include <files.h>
```

Data Fields

- sem_t * pid
- sem_t * tgt
- sem_t * vot
- sem_t * win

4.4.1 Detailed Description

Define el struct de mutexes que se usan en el proyecto.

4.4.2 Field Documentation

4.4.2.1 pid

```
sem_t* Miner_Mutexes::pid
```

Mutex que protege el fichero de PIDs

4.4.2.2 tgt

```
sem_t* Miner_Mutexes::tgt
```

Mutex que protege el fichero de target

4.4.2.3 vot

```
sem_t* Miner_Mutexes::vot
```

Mutex que protege el fichero de votos

4.4.2.4 win

```
sem_t* Miner_Mutexes::win
```

Mutex que protege las secuencias del ganador de ronda

The documentation for this struct was generated from the following file:

- files.h

Chapter 5

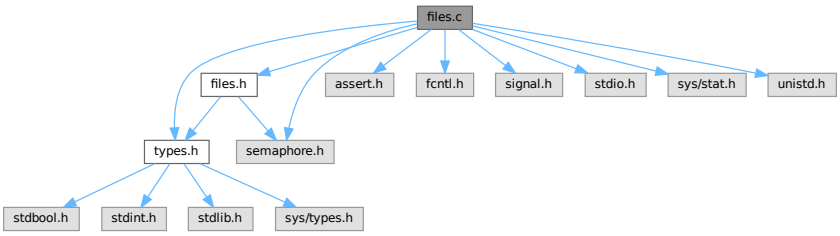
File Documentation

5.1 files.c File Reference

Implementacion de manejo de archivos.

```
#include "files.h"
#include "types.h"
#include <assert.h>
#include <fcntl.h>
#include <semaphore.h>
#include <signal.h>
#include <stdio.h>
#include <sys/stat.h>
#include <unistd.h>
```

Include dependency graph for files.c:



Functions

- void initialize_mutexes (Miner_Mutexes ∗sems)

  Inicializa los semaforos a 1 Si no existian, los crea, en caso contrario, obtiene la direccion de los semaforos ya creados.
- void **close_mutexes** (Miner_Mutexes ∗sems)

  Libera la memoria de los semaforos.
- void open_pipes (i32 ∗miner_pipe, i32 ∗logger_pipe)

  Apertura de tuberias.
- void close_pipes (i32 ∗miner_pipe, i32 ∗logger_pipe)

Cierre de tuberias.

- i32 write_pid_unlocked (const char ∗filename)

  Apunta el pid en el fichero de PIDs.
- i32 read_target_unlocked (const char ∗filename, u64 ∗out_target)

  Lee el objetivo desde el fichero.
- i32 write_target_unlocked (const char ∗filename, u64 target)

  Escribe un nuevo objetivo en el fichero.
- i32 get_active_pids_unlocked (const char ∗filename, pid_t ∗active_miners, pid_t removed_pid, bool cleanup)

  Obtiene los pids activos y limpia los inactivos del fichero.
- void write_vote (const char ∗filename, char vote)

  Escribe el voto del proceso en el fichero.
- bool count_votes (const char ∗filename, pid_t winner_pid, u32 ∗out_positives, u32 ∗total)

  Hace el recuento de votos de la solucion.

5.1.1 Detailed Description

Implementacion de manejo de archivos.

Author

Danyyil Shykerynets

- Implementa las funciones necesarias para lectura y escritura en los ficheros del proyecto.

Version

1.0

Date

2026-04-02

Copyright

(c) 2026 Author. All Rights Reserved.

5.1.2 Function Documentation

5.1.2.1 close_pipes()

```
void close_pipes (
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Cierre de tuberias.

Parameters

miner_pipe	Tuberia minero—>registrador
logger_pipe	Tuberia registrador—>minero

5.1.2.2 count_votes()

```
bool count_votes (
            const char * filename,
            pid_t winner_pid,
            u32 * out_positives,
            u32 * total )
```

Hace el recuento de votos de la solucion.

Parameters

filename	Nombre de fichero con votos del resto
winner_pid	PID del proceso ganador de la ronda
out_positives	Puntero en el que se guardan el numero de positivos
total	Puntero donde almacenar total de votos para logger

Returns

TRUE si la solucion ha sido aceptada, FALSE en caso contrario

5.1.2.3 get_active_pids_unlocked()

```
i32 get_active_pids_unlocked (
            const char * filename,
            pid_t * active_miners,
            pid_t removed_pid,
            bool cleanup )
```

Obtiene los pids activos y limpia los inactivos del fichero.

Parameters

filename	Nombre del fichero con PIDS
active_miners	Puntero a array donde guardar los pids activos (No reserva memoria)
removed_pid	PID del proceso a eliminar de los activos (en general getpid())
cleanup	Boolean para eliminar removed_pid si TRUE

Returns

Devuelve el numero de procesos activos del fichero o ERR en caso de fallo

5.1.2.4 initialize_mutexes()

```
void initialize_mutexes (
            Miner_Mutexes * sems )
```

Inicializa los semaforos a 1 Si no existian, los crea, en caso contrario, obtiene la direccion de los semaforos ya creados.

Returns

Abre los semaforos de una estructura Miner_Mutexes

5.1.2.5 open_pipes()

```
void open_pipes (
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Apertura de tuberias.

Parameters

miner_pipe	Tuberia minero—>registrador
logger_pipe	Tuberia registrador-—>minero En caso de error termina el programa con die()

5.1.2.6 read_target_unlocked()

```
i32 read_target_unlocked (
            const char * filename,
            u64 * out_target )
```

Lee el objetivo desde el fichero.

Parameters

filename	Nombre del fichero con TARGET
out_target	Puntero de valor de retorno (Target)

Returns

OK en caso de exito, ERR en caso contrario

5.1.2.7 write_pid_unlocked()

```
i32 write_pid_unlocked (
            const char * filename )
```

Apunta el pid en el fichero de PIDs.

Parameters

filename	Nombre del fichero
mutex_pid	Mutex que protege el fichero

Returns

OK en caso de EXITO, ERR si algo ha salido mal

5.1.2.8 write_target_unlocked()

```
i32 write_target_unlocked (
            const char * filename,
            u64 target )
```

Escribe un nuevo objetivo en el fichero.

Parameters

filename	Nombre del fichero con TARGET
target	Valor numérico del objetivo a escribir

Returns

OK en caso de exito, ERR en caso de fallo

5.1.2.9 write_vote()

```
void write_vote (
            const char * filename,
            char vote )
```

Escribe el voto del proceso en el fichero.

Parameters

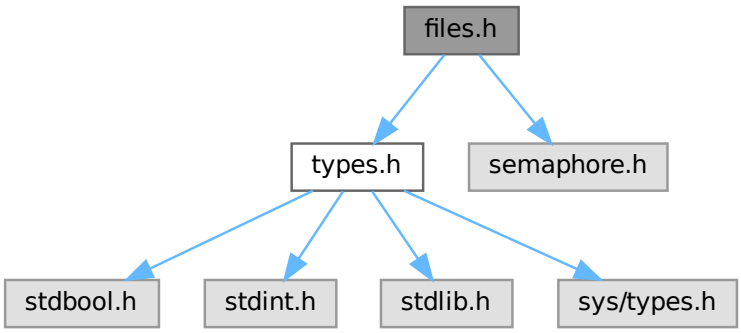
filename	Nombre del fichero con votos
vote	Char con voto (Y or N)

5.2 files.h File Reference

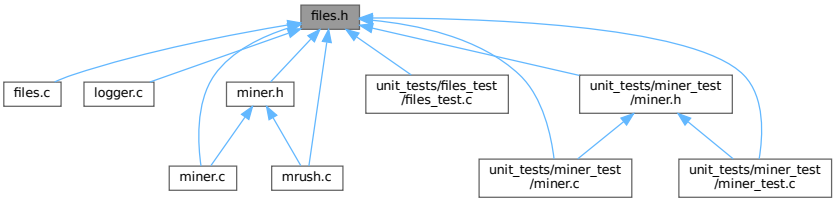
Interfaz de manejo de archivos.

```
#include "types.h"
#include <semaphore.h>
```

Include dependency graph for files.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct Miner_Mutexes

  Define el struct de mutexes que se usan en el proyecto.

Macros

Extremos de tuberías (Pipes)

- #define READ 0
- #define WRITE 1

Nombres de ficheros

- #define TARGET_FILE "Target.tgt"
- #define PID_FILE "PIDs.pid"
- #define VOTES_FILE "Voting.vot"

Semaforos de ficheros

- #define TARGET_MUTEX "/sem_miner_target"
- #define PID_MUTEX "/sem_miner_pid"
- #define VOTES_MUTEX "/sem_miner_votes"
- #define WINNER_MUTEX "/sem_miner_winner"

Functions

- void initialize_mutexes (Miner_Mutexes ∗sems)

  Inicializa los semaforos a 1 Si no existian, los crea, en caso contrario, obtiene la direccion de los semaforos ya creados.
- void **close_mutexes** (Miner_Mutexes ∗sems)

  Libera la memoria de los semaforos.
- void open_pipes (i32 ∗miner_pipe, i32 ∗logger_pipe)

  Apertura de tuberias.
- void close_pipes (i32 ∗miner_pipe, i32 ∗logger_pipe)

  Cierre de tuberias.
- i32 write_pid_unlocked (const char ∗filename)

  Apunta el pid en el fichero de PIDs.
- i32 read_target_unlocked (const char ∗filename, u64 ∗out_target)

  Lee el objetivo desde el fichero.
- i32 write_target_unlocked (const char ∗filename, u64 target)

  Escribe un nuevo objetivo en el fichero.
- i32 get_active_pids_unlocked (const char ∗filename, pid_t ∗active_miners, pid_t removed_pid, bool cleanup)

  Obtiene los pids activos y limpia los inactivos del fichero.
- void write_vote (const char ∗filename, char vote)

  Escribe el voto del proceso en el fichero.
- bool count_votes (const char ∗filename, pid_t winner_pid, u32 ∗out_positives, u32 ∗total)

  Hace el recuento de votos de la solucion.

5.2.1 Detailed Description

Interfaz de manejo de archivos.

Author

Danyyil Shykerynets

- Contiene las funciones necesarias para lectura y escritura en los ficheros del proyecto.

Version

1.0

Date

2026-04-02

Copyright

(c) 2026 Author. All Rights Reserved.

5.2.2 Macro Definition Documentation

5.2.2.1 PID_FILE

```
#define PID_FILE "PIDs.pid"
```

Fichero con los PIDs de los mineros

5.2.2.2 PID_MUTEX

```
#define PID_MUTEX "/sem_miner_pid"
```

Mutex de fichero de pids de mineros

5.2.2.3 READ

```
#define READ 0
```

Índice del descriptor de archivo para lectura

5.2.2.4 TARGET_FILE

```
#define TARGET_FILE "Target.tgt"
```

Fichero con los objetivos del minero

5.2.2.5 TARGET_MUTEX

```
#define TARGET_MUTEX "/sem_miner_target"
```

Mutex de fichero de objetivo

5.2.2.6 VOTES_FILE

```
#define VOTES_FILE "Voting.vot"
```

Fichero con registro de votos

5.2.2.7 VOTES_MUTEX

```
#define VOTES_MUTEX "/sem_miner_votes"
```

Mutex de fichero de votos

5.2.2.8 WINNER_MUTEX

```
#define WINNER_MUTEX "/sem_miner_winner"
```

Mutex para ganador de ronda

5.2.2.9 WRITE

```
#define WRITE 1
```

Índice del descriptor de archivo para escritura

5.2.3 Function Documentation

5.2.3.1 close_pipes()

```
void close_pipes (
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Cierre de tuberias.

Parameters

miner_pipe	Tuberia minero—>registrador
logger_pipe	Tuberia registrador—>minero

5.2.3.2 count_votes()

```
bool count_votes (
            const char * filename,
            pid_t winner_pid,
            u32 * out_positives,
            u32 * total )
```

Hace el recuento de votos de la solucion.

Parameters

filename	Nombre de fichero con votos del resto
winner_pid	PID del proceso ganador de la ronda
out_positives	Puntero en el que se guardan el numero de positivos
total	Puntero donde almacenar total de votos para logger

Returns

TRUE si la solucion ha sido aceptada, FALSE en caso contrario

5.2.3.3 get_active_pids_unlocked()

```
i32 get_active_pids_unlocked (
            const char * filename,
            pid_t * active_miners,
            pid_t removed_pid,
            bool cleanup )
```

Obtiene los pids activos y limpia los inactivos del fichero.

Parameters

filename	Nombre del fichero con PIDS
active_miners	Puntero a array donde guardar los pids activos (No reserva memoria)
removed_pid	PID del proceso a eliminar de los activos (en general getpid())
cleanup	Boolean para eliminar removed_pid si TRUE

Returns

Devuelve el numero de procesos activos del fichero o ERR en caso de fallo

5.2.3.4 initialize_mutexes()

```
void initialize_mutexes (
            Miner_Mutexes * sems )
```

Inicializa los semaforos a 1 Si no existian, los crea, en caso contrario, obtiene la direccion de los semaforos ya creados.

Returns

Abre los semaforos de una estructura Miner_Mutexes

5.2.3.5 open_pipes()

```
void open_pipes (
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Apertura de tuberias.

Parameters

miner_pipe	Tuberia minero—>registrador
logger_pipe	Tuberia registrador—>minero En caso de error termina el programa con die()

5.2.3.6 read_target_unlocked()

```
i32 read_target_unlocked (
            const char * filename,
            u64 * out_target )
```

Lee el objetivo desde el fichero.

Parameters

filename	Nombre del fichero con TARGET
out_target	Puntero de valor de retorno (Target)

Returns

OK en caso de exito, ERR en caso contrario

5.2.3.7 write_pid_unlocked()

```
i32 write_pid_unlocked (
            const char * filename )
```

Apunta el pid en el fichero de PIDs.

Parameters

filename	Nombre del fichero
mutex_pid	Mutex que protege el fichero

Returns

OK en caso de EXITO, ERR si algo ha salido mal

5.2.3.8 write_target_unlocked()

```
i32 write_target_unlocked (
            const char * filename,
            u64 target )
```

Escribe un nuevo objetivo en el fichero.

Parameters

filename	Nombre del fichero con TARGET
target	Valor numérico del objetivo a escribir

Returns

OK en caso de exito, ERR en caso de fallo

5.2.3.9 write_vote()

```
void write_vote (
            const char * filename,
            char vote )
```

Escribe el voto del proceso en el fichero.

Parameters

filename	Nombre del fichero con votos
vote	Char con voto (Y or N)

5.3 files.h

Go to the documentation of this file.

```
00001
00013 #ifndef _FILES_H
00014 #define _FILES_H
00015
00016 #include "types.h"
```

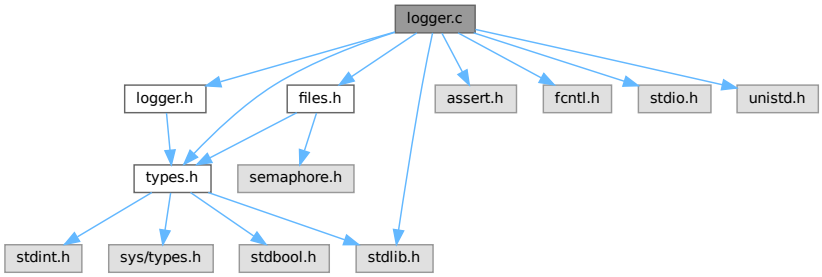
```
#include <semaphore.h>

#define READ 0
#define WRITE 1
#define TARGET_FILE "Target.tgt"
#define PID_FILE "PIDs.pid"
#define VOTES_FILE "Voting.vot"
#define TARGET_MUTEX "/sem_miner_target"
#define PID_MUTEX "/sem_miner_pid"
#define VOTES_MUTEX "/sem_miner_votes"
#define WINNER_MUTEX "/sem_miner_winner"
typedef struct {
  sem_t *pid;
  sem_t *tgt;
  sem_t *vot;
  sem_t *win;
} Miner_Mutexes;

void initialize_mutexes(Miner_Mutexes *sems);

void close_mutexes(Miner_Mutexes *sems);

void open_pipes(i32 *miner_pipe, i32 *logger_pipe);

void close_pipes(i32 *miner_pipe, i32 *logger_pipe);

i32 write_pid_unlocked(const char *filename);

i32 read_target_unlocked(const char *filename, u64 *out_target);

i32 write_target_unlocked(const char *filename, u64 target);

i32 get_active_pids_unlocked(const char *filename, pid_t *active_miners,
                             pid_t removed_pid, bool cleanup);

void write_vote(const char *filename, char vote);

bool count_votes(const char *filename, pid_t winner_pid, u32 *out_positives,
                 u32 *total);

#endif
```

5.4 logger.c File Reference

Implementación de registrador.

```
#include "logger.h"
#include "files.h"
#include "types.h"
#include <assert.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
```

Include dependency graph for logger.c:



Functions

- i32 logger (i32 ∗miner_pipe, i32 ∗logger_pipe)
  Registra los datos del minero en un fichero.

5.4.1 Detailed Description

Implementación de registrador.

Author

Fernando Blanco & Danyyil Shykerynets

- Contiene la implementación y lógica de registro en fichero de disco.

Version

1.0

Date

2026-02-25

Copyright

(c) 2026 Author. All Rights Reserved.

5.4.2 Function Documentation

5.4.2.1 logger()

```
i32 logger (
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Registra los datos del minero en un fichero.

- Se espera a leer informacion del minero a traves de miner_pipe. A continuación se escribe la informacion en el fichero con formato indicado en el enunciado. Por último se indica al minero que continue buscando la solucion de la siguiente ronda

Parameters

miner_pipe	Tuberia de escucha del minero
logger_pipe	Tuberia de escritura AL minero

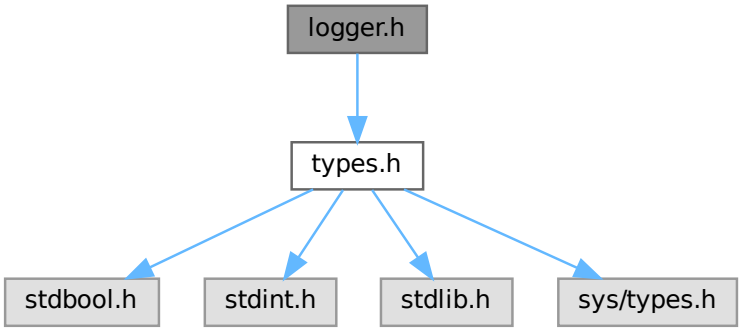
Returns

Devuelve EXIT_SUCCESS en caso de exito, EXIT_FAILURE en caso contrario

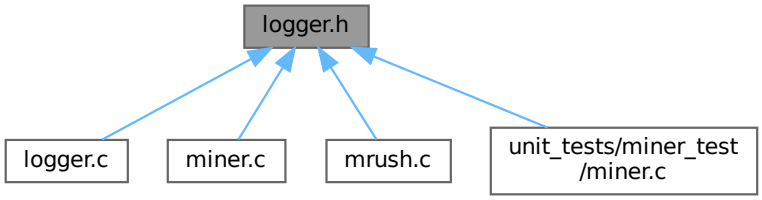
5.5 logger.h File Reference

Interfaz de registrador.

`#include "types.h"`
Include dependency graph for logger.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct Logger_args

  Argumentos y datos del bloque minado Empleamos esta estructura para manejar los argumentos del registrador Son los argumentos que almacenara el registrador en el fichero.

Functions

- i32 logger (i32 ∗miner_pipe, i32 ∗logger_pipe)

  Registra los datos del minero en un fichero.

5.5.1 Detailed Description

Interfaz de registrador.

Author

Fernando Blanco & Danyyil Shykerynets

- Contiene las funciones necesarias para inicializar el registrador y escribir el registro de soluciones en el archivo

Version

1.0

Date

2026-02-25

Copyright

(c) 2026 Author. All Rights Reserved.

5.5.2 Function Documentation

5.5.2.1 logger()

```
i32 logger (
            i32 * miner_pipe,
            i32 * logger_pipe )
```

Registra los datos del minero en un fichero.

- Se espera a leer informacion del minero a traves de miner_pipe. A continuación se escribe la informacion en el fichero con formato indicado en el enunciado. Por último se indica al minero que continue buscando la solucion de la siguiente ronda

Parameters

miner_pipe	Tuberia de escucha del minero
logger_pipe	Tuberia de escritura AL minero

Returns

Devuelve EXIT_SUCCESS en caso de exito, EXIT_FAILURE en caso contrario

5.6 logger.h

Go to the documentation of this file.

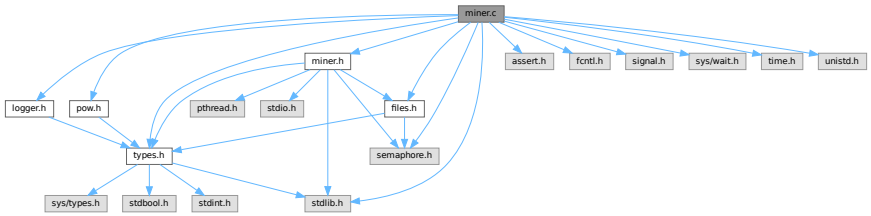
```
#ifndef _LOGGER_H
#define _LOGGER_H

#include "types.h"

typedef struct {
  u64 id;
  u64 target;
  u64 solution;
  u64 pos_votes;
  u64 votes;
  u64 wallets;
  pid_t winner;
  i32 validated;
} Logger_args;

/* Dejamos pid_t(4bytes) y validated(4 Byte) para el final para minimizar
 * padding y alinear struct a 56 bytes. Podriamos usar tipos de menor tamaño
 * para validated, pero con 4 bytes conseguimos 0 bytes de padding */

i32 logger(i32 *miner_pipe, i32 *logger_pipe);

#endif
```

5.7 miner.c File Reference

Implementación de minero.

```
#include "miner.h"
#include "files.h"
#include "logger.h"
#include "pow.h"
#include "types.h"
#include <assert.h>
#include <fcntl.h>
#include <semaphore.h>
#include <signal.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>
```

Include dependency graph for miner.c:



Macros

- #define TARGET_INIT 0

Functions

- char ∗ get_time_str ()

  Obtiene la hora actual en formato HH:MM:SS.
- void comunicar_logger (i32 ∗miner_pipe, Logger_args ∗args)

  Manda un mensaje a logger Función auxiliar para minero.
- void exit_network (const char ∗filename, Miner_Mutexes ∗sems)

  El minero abandona la red Abandona la red, libera la memoria dedicada a los semaforos por el proceso, y si es el ultimo, elimina todas las referencias a los semaforos para que no queden activos en shm.
- void handler (int sig)

  Manejador de señales Coloca el flag de timeout a 1, indicando que el minero ha terminado Coloca el flag de start↩ _mining a 1, comenzando ronda de mineria Coloca el flag de start_voting a 1, comenzando ronda de votacion.
- void **setup_signals** ()

  Hace el setup de las señales Simplemente enlaza el handler al struct de acciones de señal.
- void miner_set_alarm (u64 seconds, timer_t ∗timer)

  Inicializa el temporizador del minero Establece el manejador de señal y crea el temporizador de forma que al terminar se envie SIGALARM. En caso de error termina el programa.
- void wait_more_miners (Miner_Mutexes ∗sems)

  Espera a que se unan mas mineros a la red En comentarios, tenemos la opcion de que la espera tambien se haga con un maximo de intentos para prevenir que el proceso se quede en stall.
- void **wait_signal** (int sig, volatile sig_atomic_t ∗cond)

  Espera a la llegada de SIGUSR1 o SIGALRM de forma segura Con sigsuspend y sigprocmask suspendemos el proceso de forma que si recibe alguna señal durante la suspension no se pierda sino que ejecute su handler una vez despierte.
- void wait_votes (u32 votes, Miner_Mutexes ∗sems)

  Hace espera inactiva mientras los procesos que no han ganado votan.
- void ∗ pow_seek (void ∗arg)

  Busca el valor objetivo en un rango dado Funcion de calculo.
- u64 calcular_solucion (u64 target, Miner_data ∗args)

  Crea los hilos y separa la tarea.
- void minero (Miner_data ∗args, i32 ∗miner_pipe, i32 ∗logger_pipe, Miner_Mutexes ∗sems)

  Gestiona toda la lógica del minero Esta funcion llama a create_threads y manda informacion a logger a traves de pipes.

Variables

- volatile sig_atomic_t timeout = 0
- volatile sig_atomic_t start_mining = 0
- volatile sig_atomic_t start_voting = 0

5.7.1 Detailed Description

Implementación de minero.

Author

Fernando Blanco & Danyyil Shykerynets

- Contiene la implementación de las funciones de cálculo del POW, así como la lógica del miner implementación de las funciones de cálculo del POW, así como la lógica del minero y el IPC entre minero y registrador

Version

2.0

Date

2026-04-01

Copyright

(c) 2026 Author. All Rights Reserved.

5.7.2 Macro Definition Documentation

5.7.2.1 TARGET_INIT

```
#define TARGET_INIT 0
```

Valor de inicializacion del target

5.7.3 Function Documentation

5.7.3.1 calcular_solucion()

```
u64 calcular_solucion (
            u64 target,
            Miner_data * args )
```

Crea los hilos y separa la tarea.

Parameters

target	Objetivo de busqueda de la ronda
miner_data	Estructura con informacion para minero (tiempo y numero de hilos)

Returns

Devuelve la solucion para el POW con objetivo args->target

5.7.3.2 comunicar_logger()

```
void comunicar_logger (
            i32 * miner_pipe,
            Logger_args * args )
```

Manda un mensaje a logger Función auxiliar para minero.

Parameters

miner_pipe	Tuberia de escritura del minero al registrador
args	Estructura de argumentos del logger

5.7.3.3 exit_network()

```
void exit_network (
            const char * filename,
            Miner_Mutexes * sems )
```

El minero abandona la red Abandona la red, libera la memoria dedicada a los semaforos por el proceso, y si es el ultimo, elimina todas las referencias a los semaforos para que no queden activos en shm.

Parameters

filename	Fichero con PIDs de toda la red
sems	Estructura de semaforos del proyecto

5.7.3.4 get_time_str()

```
char * get_time_str ( )
```

Obtiene la hora actual en formato HH:MM:SS.

Returns

Puntero a un string estatico con la hora

5.7.3.5 handler()

```
void handler (
            int sig )
```

Manejador de señales Coloca el flag de timeout a 1, indicando que el minero ha terminado Coloca el flag de start_mining a 1, comenzando ronda de mineria Coloca el flag de start_voting a 1, comenzando ronda de votacion.

Parameters

sig	Numero de la señal a manejar (SIGALARM, SIGUSR1 o SIGUSR2)

5.7.3.6 miner_set_alarm()

```
void miner_set_alarm (
            u64 seconds,
            timer_t * timer )
```

Inicializa el temporizador del minero Establece el manejador de señal y crea el temporizador de forma que al terminar se envie SIGALARM. En caso de error termina el programa.

Parameters

seconds	Segundos de vida del minero
timer	Puntero a objeto timer_t

5.7.3.7 minero()

```
void minero (
            Miner_data * args,
            i32 * miner_pipe,
            i32 * logger_pipe,
            Miner_Mutexes * sems )
```

Gestiona toda la lógica del minero Esta funcion llama a create_threads y manda informacion a logger a traves de pipes.

Parameters

miner_data	Estructura con informacion para minero (rondas, target y numero de hilos)
miner_pipe	Tuberia minero—>registrador
logger_pipe	Tuberia registrador-—>minero
sems	Estructura de semaforos del sistema

5.7.3.8 pow_seek()

```
void * pow_seek (
            void * arg )
```

Busca el valor objetivo en un rango dado Funcion de calculo.

Parameters

arg	Puntero a lista de argumentos

Returns

Devuelve puntero al numero que consigue el valor objetivo

5.7.3.9 wait_more_miners()

```
void wait_more_miners (
            Miner_Mutexes * sems )
```

Espera a que se unan mas mineros a la red En comentarios, tenemos la opcion de que la espera tambien se haga con un maximo de intentos para prevenir que el proceso se quede en stall.

Espera a que se unan mas mineros a la red.

Parameters

sems	Estructura de semaforos del sistema

5.7.3.10 wait_votes()

```
void wait_votes (
            u32 votes,
            Miner_Mutexes * sems )
```

Hace espera inactiva mientras los procesos que no han ganado votan.

Parameters

votes	Numero de votos esperados
sems	Semaforos del sistema

5.7.4 Variable Documentation

5.7.4.1 start_mining

```
volatile sig_atomic_t start_mining = 0
```

Flag de comienzo de mineria Mientras sea 0, los mineros esperan Una vez a 1, comienza la ronda de mineria Se reseteara a 0 tras el comienzo de la ronda

5.7.4.2 start_voting

```
volatile sig_atomic_t start_voting = 0
```

Flag de comienzo de votacion Mientras este a 0, los mineros esperan y no votan Cuando se ponga a 1, comienza la ronda de votacion Se reseteara a 0 tras el comienzo de las votaciones

5.7.4.3 timeout

```
volatile sig_atomic_t timeout = 0
```

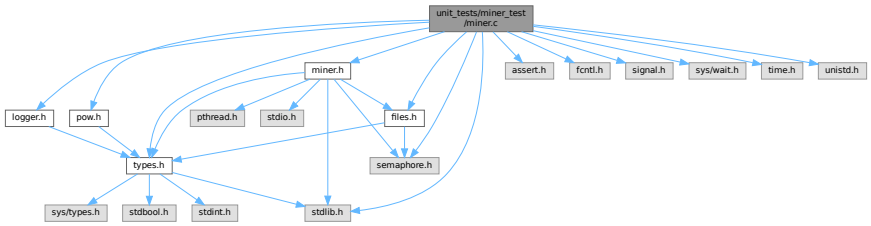
Flag de timeout del minero Mientras sea 0, el minero trabaja, en cuanto se acabe el tiempo y se reciba SIGALARM, se pone a 1 y el minero abandona la red

5.8 unit_tests/miner_test/miner.c File Reference

Implementación de minero.

```
#include "miner.h"
#include "files.h"
#include "logger.h"
#include "pow.h"
#include "types.h"
#include <assert.h>
#include <fcntl.h>
#include <semaphore.h>
#include <signal.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>
```

Include dependency graph for miner.c:



Macros

- #define TARGET_INIT 0

Functions

- char ∗ get_time_str ()

  Obtiene la hora actual en formato HH:MM:SS.
- void comunicar_logger (i32 ∗miner_pipe, Logger_args ∗args)

  Manda un mensaje a logger Función auxiliar para minero.
- void exit_network (const char ∗filename, Miner_Mutexes ∗sems)

  El minero abandona la red Abandona la red, libera la memoria dedicada a los semaforos por el proceso, y si es el ultimo, elimina todas las referencias a los semaforos para que no queden activos en shm.
- void handler (int sig)

  Manejador de señales Coloca el flag de timeout a 1, indicando que el minero ha terminado Coloca el flag de start↩ _mining a 1, comenzando ronda de mineria Coloca el flag de start_voting a 1, comenzando ronda de votacion.
- void **setup_signals** ()

  Hace el setup de las señales Simplemente enlaza el handler al struct de acciones de señal.
- void miner_set_alarm (u64 seconds, timer_t ∗timer)

  Inicializa el temporizador del minero Establece el manejador de señal y crea el temporizador de forma que al terminar se envie SIGALARM. En caso de error termina el programa.
- void wait_more_miners (Miner_Mutexes ∗sems)

Espera a que se unan mas mineros a la red En comentarios, tenemos la opcion de que la espera tambien se haga con un maximo de intentos para prevenir que el proceso se quede en stall.

- void **wait_signal** (int sig, volatile sig_atomic_t ∗cond)

  Espera a la llegada de SIGUSR1 o SIGALRM de forma segura Con sigsuspend y sigprocmask suspendemos el proceso de forma que si recibe alguna señal durante la suspension no se pierda sino que ejecute su handler una vez despierte.
- void wait_votes (u32 votes, Miner_Mutexes ∗sems)

  Hace espera inactiva mientras los procesos que no han ganado votan.
- void ∗ pow_seek (void ∗arg)

  Busca el valor objetivo en un rango dado Funcion de calculo.
- u64 calcular_solucion (u64 target, Miner_data ∗args)

  Crea los hilos y separa la tarea.
- void minero (Miner_data ∗args, i32 ∗miner_pipe, i32 ∗logger_pipe, Miner_Mutexes ∗sems)

  Gestiona toda la lógica del minero Esta funcion llama a create_threads y manda informacion a logger a traves de pipes.

Variables

- volatile sig_atomic_t timeout = 0
- volatile sig_atomic_t start_mining = 0
- volatile sig_atomic_t start_voting = 0

5.8.1 Detailed Description

Implementación de minero.

Author

Fernando Blanco & Danyyil Shykerynets

- Contiene la implementación de las funciones de cálculo del POW, así como la lógica del miner implementación de las funciones de cálculo del POW, así como la lógica del minero y el IPC entre minero y registrador

Version

2.0

Date

2026-04-01

Copyright

(c) 2026 Author. All Rights Reserved.

5.8.2 Macro Definition Documentation

5.8.2.1 TARGET_INIT

```
#define TARGET_INIT 0
```

Valor de inicializacion del target

5.8.3 Function Documentation

5.8.3.1 calcular_solucion()

```
u64 calcular_solucion (
            u64 target,
            Miner_data * args )
```

Crea los hilos y separa la tarea.

Parameters

target	Objetivo de busqueda de la ronda
miner_data	Estructura con informacion para minero (tiempo y numero de hilos)

Returns

Devuelve la solucion para el POW con objetivo args->target

5.8.3.2 comunicar_logger()

```
void comunicar_logger (
            i32 * miner_pipe,
            Logger_args * args )
```

Manda un mensaje a logger Función auxiliar para minero.

Parameters

miner_pipe	Tuberia de escritura del minero al registrador
args	Estructura de argumentos del logger

5.8.3.3 exit_network()

```
void exit_network (
            const char * filename,
            Miner_Mutexes * sems )
```

El minero abandona la red Abandona la red, libera la memoria dedicada a los semaforos por el proceso, y si es el ultimo, elimina todas las referencias a los semaforos para que no queden activos en shm.

Parameters

filename	Fichero con PIDs de toda la red
sems	Estructura de semaforos del proyecto

5.8.3.4 get_time_str()

```
char * get_time_str ( )
```

Obtiene la hora actual en formato HH:MM:SS.

Returns

Puntero a un string estatico con la hora

5.8.3.5 handler()

```
void handler (
            int sig )
```

Manejador de señales Coloca el flag de timeout a 1, indicando que el minero ha terminado Coloca el flag de start_mining a 1, comenzando ronda de mineria Coloca el flag de start_voting a 1, comenzando ronda de votacion.

Parameters

sig	Numero de la señal a manejar (SIGALARM, SIGUSR1 o SIGUSR2)

5.8.3.6 miner_set_alarm()

```
void miner_set_alarm (
            u64 seconds,
            timer_t * timer )
```

Inicializa el temporizador del minero Establece el manejador de señal y crea el temporizador de forma que al terminar se envie SIGALARM. En caso de error termina el programa.

Parameters

seconds	Segundos de vida del minero
timer	Puntero a objeto timer_t

5.8.3.7 minero()

```
void minero (
            Miner_data * args,
            i32 * miner_pipe,
            i32 * logger_pipe,
            Miner_Mutexes * sems )
```

Gestiona toda la lógica del minero Esta funcion llama a create_threads y manda informacion a logger a traves de pipes.

Parameters

miner_data	Estructura con informacion para minero (rondas, target y numero de hilos)
miner_pipe	Tuberia minero—>registrador
logger_pipe	Tuberia registrador-—>minero
sems	Estructura de semaforos del sistema

5.8.3.8 pow_seek()

```
void * pow_seek (
            void * arg )
```

Busca el valor objetivo en un rango dado Funcion de calculo.

Parameters

arg	Puntero a lista de argumentos

Returns

Devuelve puntero al numero que consigue el valor objetivo

5.8.3.9 wait_more_miners()

```
void wait_more_miners (
            Miner_Mutexes * sems )
```

Espera a que se unan mas mineros a la red En comentarios, tenemos la opcion de que la espera tambien se haga con un maximo de intentos para prevenir que el proceso se quede en stall.

Espera a que se unan mas mineros a la red.

Parameters

sems	Estructura de semaforos del sistema

5.8.3.10 wait_votes()

```
void wait_votes (
            u32 votes,
            Miner_Mutexes * sems )
```

Hace espera inactiva mientras los procesos que no han ganado votan.

Parameters

votes	Numero de votos esperados
sems	Semaforos del sistema

5.8.4 Variable Documentation

5.8.4.1 start_mining

```
volatile sig_atomic_t start_mining = 0
```

Flag de comienzo de mineria Mientras sea 0, los mineros esperan Una vez a 1, comienza la ronda de mineria Se reseteara a 0 tras el comienzo de la ronda

5.8.4.2 start_voting

```
volatile sig_atomic_t start_voting = 0
```

Flag de comienzo de votacion Mientras este a 0, los mineros esperan y no votan Cuando se ponga a 1, comienza la ronda de votacion Se reseteara a 0 tras el comienzo de las votaciones

5.8.4.3 timeout

```
volatile sig_atomic_t timeout = 0
```

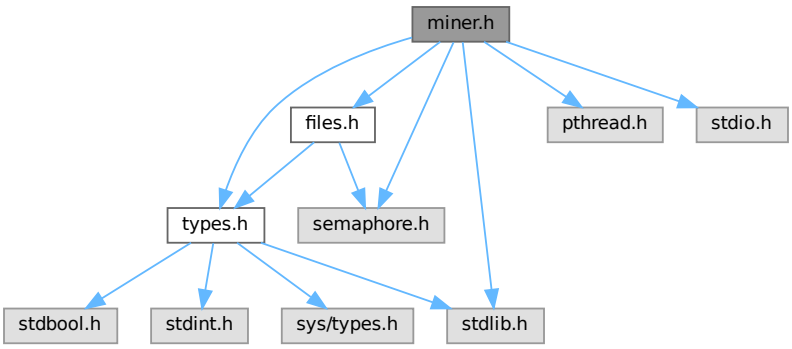
Flag de timeout del minero Mientras sea 0, el minero trabaja, en cuanto se acabe el tiempo y se reciba SIGALARM, se pone a 1 y el minero abandona la red

5.9 miner.h File Reference

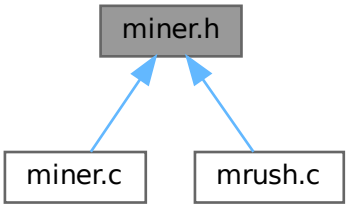
Interfaz de mineros.

```
#include "files.h"
#include "types.h"
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for miner.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct Arg_hilos

  Estructura para manejo de argumentos de hilos Servira para guardar valores para busqueda de hash a traves de POW.
- struct Miner_data

  Estructura para manejo de argumentos de Minero Servira para guardar los datos relevantes para el minero.

Functions

- void minero (Miner_data ∗args, i32 ∗miner_pipe, i32 ∗logger_pipe, Miner_Mutexes ∗sems)

  Gestiona toda la lógica del minero Esta funcion llama a create_threads y manda informacion a logger a traves de pipes.

5.9.1 Detailed Description

Interfaz de mineros.

Author

Fernando Blanco & Danyyil Shykerynets

- Contiene las declaración de funciones y estructuras necesarios para el minero y las tuberias así como las funciones de cálculo

Version

1.0

Date

2026-02-12

5.9.2 Function Documentation

5.9.2.1 minero()

```
void minero (
            Miner_data * args,
            i32 * miner_pipe,
            i32 * logger_pipe,
            Miner_Mutexes * sems )
```

Gestiona toda la lógica del minero Esta funcion llama a create_threads y manda informacion a logger a traves de pipes.

Parameters

miner_data	Estructura con informacion para minero (rondas, target y numero de hilos)
miner_pipe	Tuberia minero—>registrador
logger_pipe	Tuberia registrador-—>minero
sems	Estructura de semaforos del sistema

5.10 miner.h

Go to the documentation of this file.

```
00001
00012 #ifndef _MINER_H
00013 #define _MINER_H
00014
00015 #include "files.h"
00016 #include "types.h"
00017 #include <pthread.h>
00018 #include <semaphore.h>
00019 #include <stdio.h>
00020 #include <stdlib.h>
00021
00022 /****************************** DATOS PUBLICOS ****************************/
00023
00029 typedef struct {
00030   u64 target;
00031   u64 min;
00032   u64 max;
00033   volatile int *found_value;
00034 } Arg_hilos;
00035
00042 typedef struct {
00043   u64 time;
00044   u64 n_threads;
00045 } Miner_data;
00046
00047 /****************************** DATOS PRIVADOS ****************************/
00048
00060 void minero(Miner_data *args, i32 *miner_pipe, i32 *logger_pipe,
00061             Miner_Mutexes *sems);
00062
00063 #endif
```

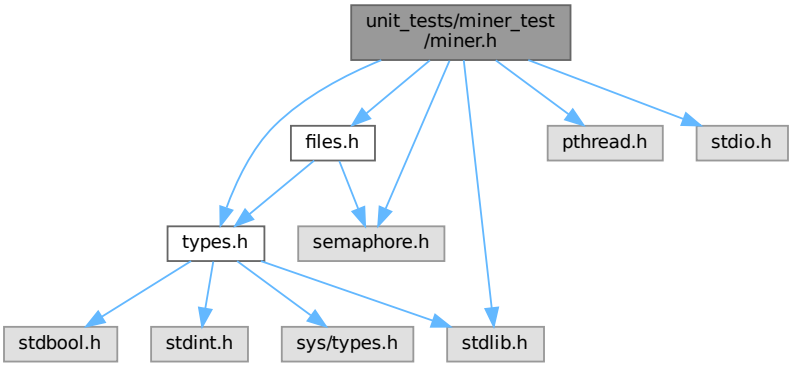
5.11 unit_tests/miner_test/miner.h File Reference

Interfaz de mineros.

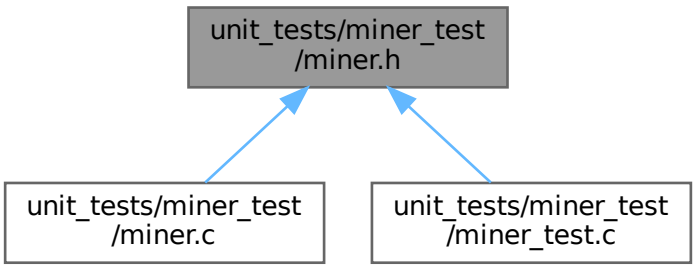
```
#include "files.h"
#include "types.h"
```

```
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for miner.h:



This graph shows which files directly or indirectly include this file:



Data Structures

- struct Arg_hilos

  Estructura para manejo de argumentos de hilos Servira para guardar valores para busqueda de hash a traves de POW.
- struct Miner_data

  Estructura para manejo de argumentos de Minero Servira para guardar los datos relevantes para el minero.

Functions

- void minero (Miner_data ∗args, i32 ∗miner_pipe, i32 ∗logger_pipe, Miner_Mutexes ∗sems)

Gestiona toda la lógica del minero Esta funcion llama a create_threads y manda informacion a logger a traves de pipes.

- void wait_more_miners (Miner_Mutexes *sems)

  Espera a que se unan mas mineros a la red.
- void wait_votes (u32 votes, Miner_Mutexes *sems)

  Hace espera inactiva mientras los procesos que no han ganado votan.
- u64 calcular_solucion (u64 target, Miner_data *args)

  Crea los hilos y separa la tarea.

5.11.1 Detailed Description

Interfaz de mineros.

Author

Fernando Blanco & Danyyil Shykerynets

- Contiene las declaración de funciones y estructuras necesarios para el minero y las tuberias así como las funciones de cálculo

Version

1.0

Date

2026-02-12

5.11.2 Function Documentation

5.11.2.1 calcular_solucion()

```
u64 calcular_solucion (
            u64 target,
            Miner_data * args )
```

Crea los hilos y separa la tarea.

Parameters

target	Objetivo de busqueda de la ronda
args	Estructura con informacion para minero (tiempo y numero de hilos)

Returns

Devuelve la solucion para el POW con objetivo

Parameters

target	Objetivo de busqueda de la ronda
miner_data	Estructura con informacion para minero (tiempo y numero de hilos)

Returns

Devuelve la solucion para el POW con objetivo args->target

5.11.2.2 minero()

```
void minero (
            Miner_data * args,
            i32 * miner_pipe,
            i32 * logger_pipe,
            Miner_Mutexes * sems )
```

Gestiona toda la lógica del minero Esta funcion llama a create_threads y manda informacion a logger a traves de pipes.

Parameters

miner_data	Estructura con informacion para minero (rondas, target y numero de hilos)
miner_pipe	Tuberia minero—>registrador
logger_pipe	Tuberia registrador-—>minero
sems	Estructura de semaforos del sistema

5.11.2.3 wait_more_miners()

```
void wait_more_miners (
            Miner_Mutexes * sems )
```

Espera a que se unan mas mineros a la red.

Parameters

sems	Estructura de semaforos del sistema

Espera a que se unan mas mineros a la red.

Parameters

sems	Estructura de semaforos del sistema

5.11.2.4 wait_votes()

```
void wait_votes (
```

```
u32 votes,
Miner_Mutexes * sems )
```

Hace espera inactiva mientras los procesos que no han ganado votan.

Parameters

votes	Numero de votos esperados
sems	Semaforos del sistema

5.12 miner.h

Go to the documentation of this file.

```
00001
00012 #ifndef _MINER_H
00013 #define _MINER_H
00014
00015 #include "files.h"
00016 #include "types.h"
00017 #include <pthread.h>
00018 #include <semaphore.h>
00019 #include <stdio.h>
00020 #include <stdlib.h>
00021
00022 /****************************** DATOS PUBLICOS ****************************/
00023
00029 typedef struct {
00030   u64 target;
00031   u64 min;
00032   u64 max;
00033   volatile int *found_value;
00034 } Arg_hilos;
00035
00042 typedef struct {
00043   u64 time;
00044   u64 n_threads;
00045 } Miner_data;
00046
00058 void minero(Miner_data *args, i32 *miner_pipe, i32 *logger_pipe,
00059             Miner_Mutexes *sems);
00060
00061 /***************************** FUNCIONES EXPUESTAS ************************/
00062 /* Expuestas en la cabecera principalmente para permitir el Testing Unitario */
00063
00068 void wait_more_miners(Miner_Mutexes *sems);
00069
00075 void wait_votes(u32 votes, Miner_Mutexes *sems);
00076
00083 u64 calcular_solucion(u64 target, Miner_data *args);
00084
00085 #endif
```

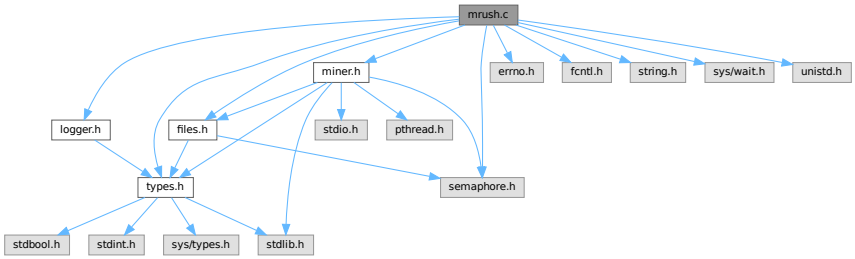
5.13 mrush.c File Reference

Punto de entrada del programa.

```
#include "files.h"
#include "logger.h"
#include "miner.h"
#include "types.h"
#include <errno.h>
#include <fcntl.h>
#include <semaphore.h>
#include <string.h>
```

```
#include <sys/wait.h>
#include <unistd.h>
```

Include dependency graph for mrush.c:



Functions

- u64 str_to_u64 (char *input)
- int **main** (int argc, char *argv[ ])

5.13.1 Detailed Description

Punto de entrada del programa.

Author

Danyyil Shykerynets

- Contiene el main a ejecutar del programa

Version

1.0

Date

2026-02-09

5.13.2 Function Documentation

5.13.2.1 str_to_u64()

```
u64 str_to_u64 (
            char * input )
```

Parsea un numero en string a unsigned int de 64 bit de forma segura

Parameters

input	String a parsear

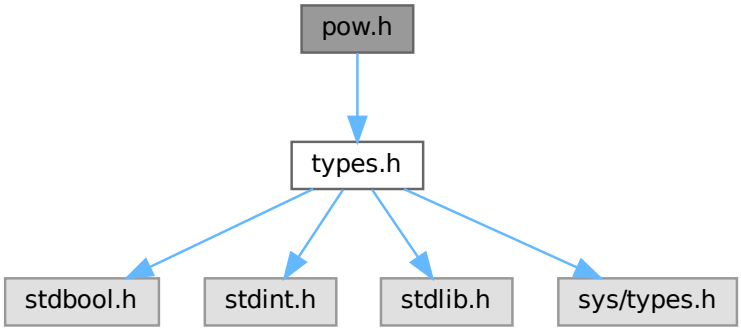
Returns

Unsigned int de 64 bit o ERR en su caso

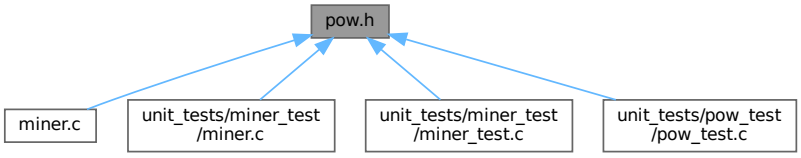
5.14 pow.h File Reference

Computation of the POW.

`#include "types.h"`
Include dependency graph for pow.h:



This graph shows which files directly or indirectly include this file:



Macros

- #define POW_LIMIT 9997697

Functions

- u64 pow_hash (u64 x)

  Computes the following hash function: f(x) = (X x + Y) % P.

5.14.1 Detailed Description

Computation of the POW.

Author

SOPER teaching team.

Version

2.0

Date

2024-02-01

Copyright

Copyright (c) 2022

5.14.2 Macro Definition Documentation

5.14.2.1 POW_LIMIT

```
#define POW_LIMIT 9997697
```

Maximum number for the hash result.

5.14.3 Function Documentation

5.14.3.1 pow_hash()

```
u64 pow_hash (
            u64 x )
```

Computes the following hash function: f(x) = (X x + Y) % P.

Parameters

x	Argument of the hash function, x.

Returns

Result of the hash function, f(x).

5.15 pow.h

Go to the documentation of this file.

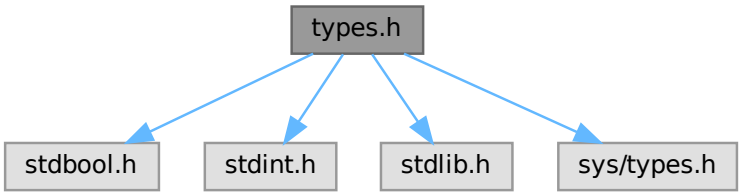
```
00001
00012 #ifndef _POW_H
00013 #define _POW_H
00014
00015 #include "types.h"
00016
00017 #define POW_LIMIT 9997697
00026 u64 pow_hash(u64 x);
00027
00028 #endif
```

5.16 types.h File Reference

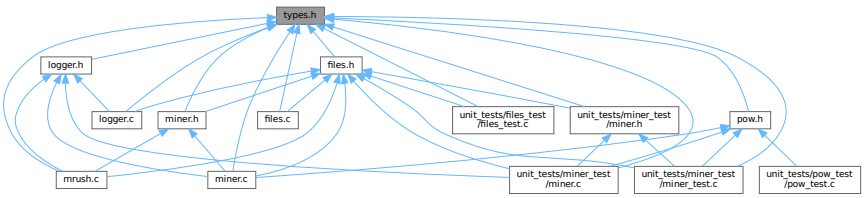
Definición de tipos de datos globales.

```
#include <stdbool.h>
#include <stdint.h>
#include <stdlib.h>
#include <sys/types.h>
```

Include dependency graph for types.h:



This graph shows which files directly or indirectly include this file:



Macros

Control de errores

- #define ERR -1
- #define OK !(ERR)
- #define die(msg)

  Imprimir error y terminar el programa.
- #define die_msg(msg)

  Termina el programa imprimiendo un mensaje lógico personalizado.

Banderas de estado

- #define FOUND 1
- #define MIN_MINERS 2
- #define MAX_TRIES 10
- #define MAX_MINERS 256

Utilidades generales

- #define MIN(a, b) ((a) < (b) ? (a) : (b))

  Devuelve el valor mínimo entre dos elementos.

Typedefs

Tipos de datos enteros y flotantes

- typedef int32_t i32
- typedef int64_t i64
- typedef uint32_t u32
- typedef uint64_t u64
- typedef float f32
- typedef double f64

5.16.1 Detailed Description

Definición de tipos de datos globales.

Author

Danyyil Shykerynets

- Este archivo contiene los alias de tipos de tamaño fijo (stdint) y las estructuras fundamentales compartidas entre el minero y el logger.

Version

1.0

Date

2026-02-09

5.16.2 Macro Definition Documentation

5.16.2.1 die

```
#define die(
            msg )
```

Value:

```
  do {                                                    \
    perror(msg);                                          \
    exit(EXIT_FAILURE);                                   \
  } while (0)
```

Imprimir error y terminar el programa.

Parameters

msg	Cadena con mensaje de error

5.16.2.2 die_msg

```
#define die_msg(
            msg )
```

Value:

```
do {                                                    \
  fprintf(stderr, "%s\n", msg);                         \
  exit(EXIT_FAILURE);                                   \
} while (0)
```

Termina el programa imprimiendo un mensaje lógico personalizado.

5.16.2.3 ERR

```
#define ERR -1
```

Código estándar de error para returns

5.16.2.4 FOUND

```
#define FOUND 1
```

Indica que un hilo ha encontrado solución

5.16.2.5 MAX_MINERS

```
#define MAX_MINERS  256
```

Define el maximo de mineros que acepto. En principio, no se hace el \ handling de que nunca supere el numero porque no es el objetivo. La \ macro solo sirve para inicializar los arrays de pids

5.16.2.6 MAX_TRIES

```
#define MAX_TRIES 10
```

Define maximo de intentos de espera inactiva

5.16.2.7 MIN

```
#define MIN(
            a,
            b ) ((a) < (b) ?  (a) :  (b))
```

Devuelve el valor mínimo entre dos elementos.

Parameters

a	Primer valor a comparar.
b	Segundo valor a comparar.

Returns

El menor de los dos valores.

5.16.2.8 MIN_MINERS

```
#define MIN_MINERS 2
```

Define el minimo de mineros para comenzar ronda

5.16.2.9 OK

```
#define OK !(ERR)
```

Código de éxito (0)

5.16.3 Typedef Documentation

5.16.3.1 f32

```
typedef float f32
```

Float 32 bit. 7 dig. precision

5.16.3.2 f64

```
typedef double f64
```

Float 64 bit. 15 dig. precision

5.16.3.3 i32

```
typedef int32_t i32
```

Entero con signo 32 bit. [-2^31, 2^31 - 1]

5.16.3.4 i64

```
typedef int64_t i64
```

Entero con signo 64 bit. [-2^63, 2^63 - 1]

5.16.3.5 u32

```
typedef uint32_t u32
```

Entero sin signo 32 bit. [0, 2^32 - 1]

5.16.3.6 u64

```
typedef uint64_t u64
```

Entero sin signo 64 bit. [0, 2^64 - 1]

5.17 types.h

Go to the documentation of this file.

```
#ifndef _TYPES_H
#define _TYPES_H

#include <stdbool.h>
#include <stdint.h>
#include <stdlib.h>
#include <sys/types.h>

typedef int32_t i32;
typedef int64_t i64;
typedef uint32_t u32;
typedef uint64_t u64;
typedef float f32;
typedef double f64;
#define ERR -1
#define OK !(ERR)
#define die(msg)                                                    \
  do {                                                              \
    perror(msg);                                                    \
    exit(EXIT_FAILURE);                                             \
  } while (0)

#define die_msg(msg)                                                \
  do {                                                              \
    fprintf(stderr, "%s\n", msg);                                   \
    exit(EXIT_FAILURE);                                             \
  } while (0)
#define FOUND 1
#define MIN_MINERS 2
#define MAX_TRIES 10
#define MAX_MINERS                                                  \
  256
#define MIN(a, b) ((a) < (b) ? (a) : (b))
#endif
```

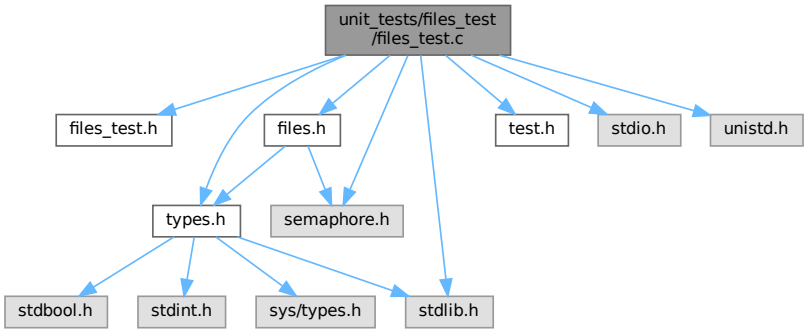
5.18 unit_tests/files_test/files_test.c File Reference

Testea el modulo de archivos.

```
#include "files_test.h"
#include "files.h"
#include "test.h"
#include "types.h"
#include <semaphore.h>
#include <stdio.h>
#include <stdlib.h>
```

```
#include <unistd.h>
```

Include dependency graph for files_test.c:



Macros

- #define **TEST_PID_FILE** "test_pids.pid"
- #define **TEST_VOTES_FILE** "test_votes.vot"
- #define **MAX_TESTS** 5

Functions

- int **main** (int argc, char **argv)
- void test1_open_pipes ()
- void test2_close_pipes ()
- void test3_mutex_init ()
- void test4_pids_io ()
- void test5_voting_system ()

5.18.1 Detailed Description

Testea el modulo de archivos.

Author

Danyyil Shykerynets & Fernando Blanco

Version

2.0

Date

2026-04-06

5.18.2 Function Documentation

5.18.2.1 test1_open_pipes()

```
void test1_open_pipes ( )
```

Test Test open_pipes

Postcondition

Los descriptores de archivo se asignan correctamente

5.18.2.2 test2_close_pipes()

```
void test2_close_pipes ( )
```

Test Test close_pipes

Postcondition

Las tuberías se cierran y los descriptores se resetean

5.18.2.3 test3_mutex_init()

```
void test3_mutex_init ( )
```

Test Test inicialización de semáforos

5.18.2.4 test4_pids_io()

```
void test4_pids_io ( )
```

Test Test lectura y escritura de PIDs

5.18.2.5 test5_voting_system()

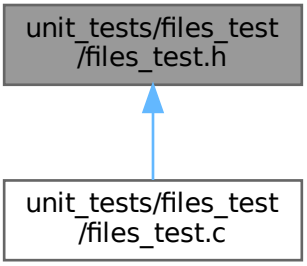
```
void test5_voting_system ( )
```

Test Test del sistema de votaciones

5.19 unit_tests/files_test/files_test.h File Reference

Declara los tests para el modulo de archivos y semaforos (files.c)

This graph shows which files directly or indirectly include this file:



Functions

- void test1_open_pipes ()
- void test2_close_pipes ()
- void test3_mutex_init ()
- void test4_pids_io ()
- void test5_voting_system ()

5.19.1 Detailed Description

Declara los tests para el modulo de archivos y semaforos (files.c)

Author

Danyyil Shykerynets & Fernando Blanco

Version

2.0

Date

2026-04-06

5.19.2 Function Documentation

5.19.2.1 test1_open_pipes()

```
void test1_open_pipes ( )
```

Test Test open_pipes

Postcondition

Los descriptores de archivo se asignan correctamente

5.19.2.2 test2_close_pipes()

```
void test2_close_pipes ( )
```

Test Test close_pipes

Postcondition

Las tuberías se cierran y los descriptores se resetean

5.19.2.3 test3_mutex_init()

```
void test3_mutex_init ( )
```

Test Test inicialización de semáforos

5.19.2.4 test4_pids_io()

```
void test4_pids_io ( )
```

Test Test lectura y escritura de PIDs

5.19.2.5 test5_voting_system()

```
void test5_voting_system ( )
```

Test Test del sistema de votaciones

5.20 files_test.h

Go to the documentation of this file.

```
#ifndef _FILES_TEST_H
#define _FILES_TEST_H

void test1_open_pipes();

void test2_close_pipes();

void test3_mutex_init();

void test4_pids_io();

void test5_voting_system();

#endif
```

5.21 logger_test.h

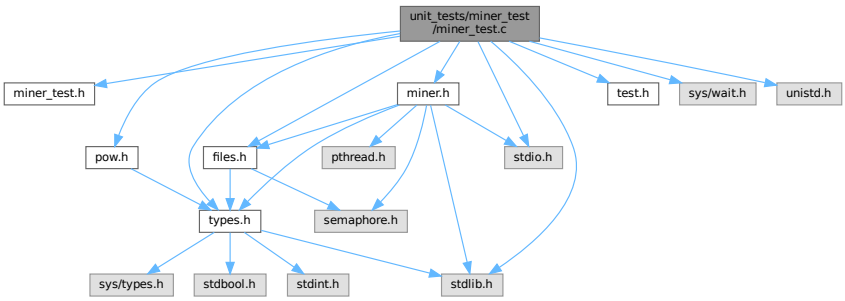
```
#ifndef _LOGGER_TEST_H
#define _LOGGER_TEST_H

void test1_logger_ipc();

#endif
```

5.22 unit_tests/miner_test/miner_test.c File Reference

Testea el modulo Miner.

```
#include "miner_test.h"
#include "files.h"
#include "miner.h"
#include "pow.h"
#include "test.h"
#include "types.h"
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>
```

Include dependency graph for miner_test.c:



Macros

- #define **MAX_TESTS** 5

Functions

- int **main** (int argc, char **argv)
- void test1_open_pipes ()
- void **test2_open_pipes** ()
- void test1_calcular_solucion ()
- void test1_wait_votes_success ()
- void test2_wait_votes_timeout ()
- void test1_wait_more_miners_success ()
- void test2_wait_more_miners_timeout ()

Variables

- volatile sig_atomic_t timeout
- volatile sig_atomic_t start_voting

5.22.1 Detailed Description

Testea el modulo Miner.

Author

Danyyil Shykerynets

Version

1.0

Date

01-03-2026 @Copyright (c) 2026 Author. All Rights Reserved.

5.22.2 Function Documentation

5.22.2.1 test1_calcular_solucion()

```
void test1_calcular_solucion ( )
```

Test Test calcular_solucion

5.22.2.2 test1_open_pipes()

```
void test1_open_pipes ( )
```

Test Test open_pipes

Postcondition

Los descriptores de archivo se asignan correctamente

5.22.2.3 test1_wait_more_miners_success()

```
void test1_wait_more_miners_success ( )
```

Test Test wait_more_miners (Success)

5.22.2.4 test1_wait_votes_success()

```
void test1_wait_votes_success ( )
```

Test Test wait_votes (Success)

5.22.2.5 test2_wait_more_miners_timeout()

```
void test2_wait_more_miners_timeout ( )
```

Test Test wait_more_miners (fracaso timeout)

Postcondition

La barrera se rompe y sale si el tiempo de vida del minero se acaba

5.22.2.6 test2_wait_votes_timeout()

```
void test2_wait_votes_timeout ( )
```

Test Test wait_votes (fracaso timeout)

Postcondition

Aborta la espera si el tiempo se agota, aunque falten votos.

5.22.3 Variable Documentation

5.22.3.1 start_voting

```
volatile sig_atomic_t start_voting [extern]
```

Flag de comienzo de votacion Mientras este a 0, los mineros esperan y no votan Cuando se ponga a 1, comienza la ronda de votacion Se reseteara a 0 tras el comienzo de las votaciones

5.22.3.2 timeout

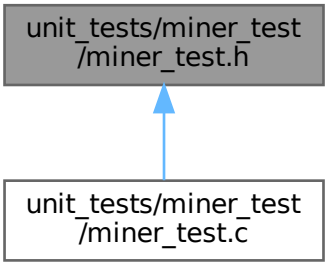
```
volatile sig_atomic_t timeout  [extern]
```

Flag de timeout del minero Mientras sea 0, el minero trabaja, en cuanto se acabe el tiempo y se reciba SIGALARM, se pone a 1 y el minero abandona la red

5.23 unit_tests/miner_test/miner_test.h File Reference

Declara los tests para el modulo miner.

This graph shows which files directly or indirectly include this file:



Functions

- void test1_calcular_solucion ()
- void test1_wait_votes_success ()
- void test2_wait_votes_timeout ()
- void test1_wait_more_miners_success ()
- void test2_wait_more_miners_timeout ()

5.23.1 Detailed Description

Declara los tests para el modulo miner.

Author

Danyyil Shykerynets

Version

1.0

Date

01-03-2025 @Copyright (c) 2026 Author. All Rights Reserved.

5.23.2 Function Documentation

5.23.2.1 test1_calcular_solucion()

```
void test1_calcular_solucion ( )
```

Test Test calcular_solucion

5.23.2.2 test1_wait_more_miners_success()

```
void test1_wait_more_miners_success ( )
```

Test Test wait_more_miners (Success)

5.23.2.3 test1_wait_votes_success()

```
void test1_wait_votes_success ( )
```

Test Test wait_votes (Success)

5.23.2.4 test2_wait_more_miners_timeout()

```
void test2_wait_more_miners_timeout ( )
```

Test Test wait_more_miners (fracaso timeout)

Postcondition

La barrera se rompe y sale si el tiempo de vida del minero se acaba

5.23.2.5 test2_wait_votes_timeout()

```
void test2_wait_votes_timeout ( )
```

Test Test wait_votes (fracaso timeout)

Postcondition

Aborta la espera si el tiempo se agota, aunque falten votos.

5.24 miner_test.h

Go to the documentation of this file.

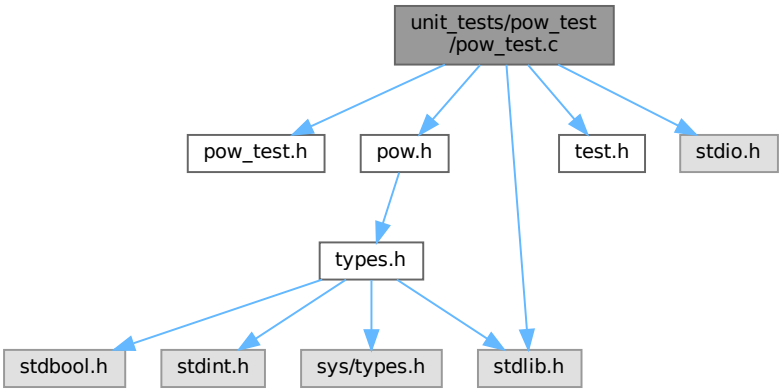
```
#ifndef _MINER_TEST_H
#define _MINER_TEST_H

void test1_calcular_solucion();

void test1_wait_votes_success();

void test2_wait_votes_timeout();

void test1_wait_more_miners_success();

void test2_wait_more_miners_timeout();

#endif
```

5.25 unit_tests/pow_test/pow_test.c File Reference

Testea el modulo pow.

```
#include "pow_test.h"
#include "pow.h"
#include "test.h"
#include <stdio.h>
#include <stdlib.h>
```

Include dependency graph for pow_test.c:



Macros

- #define **MAX_TESTS** 3

Functions

- int main (int argc, char **argv)

  Main function for POW unit tests.
- void test1_pow ()
- void **test2_pow** ()
- void **test3_pow** ()

5.25.1 Detailed Description

Testea el modulo pow.

Author

Danyyil Shykerynets

Version

1.0

Date

01-03-2026 @Copyright (c) 2026 Author. All Rights Reserved.

5.25.2 Function Documentation

5.25.2.1 main()

```
int main (
            int argc,
            char ** argv )
```

Main function for POW unit tests.

You may execute ALL or a SINGLE test 1.- No parameter -> ALL test are executed 2.- A number means a particular test (the one identified by that number) is executed

5.25.2.2 test1_pow()

```
void test1_pow ( )
```

Test Test pow function

Precondition

Long int

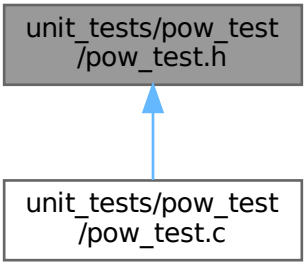
Postcondition

Hashed value

5.26 unit_tests/pow_test/pow_test.h File Reference

Declara los tests para la funcion POW.

This graph shows which files directly or indirectly include this file:



Functions

- void test1_pow ()
- void **test2_pow** ()
- void **test3_pow** ()

5.26.1 Detailed Description

Declara los tests para la funcion POW.

Author

Danyyil Shykerynets

Version

1.0

Date

01-03-2025 @Copyright (c) 2026 Author. All Rights Reserved.

5.26.2 Function Documentation

5.26.2.1 test1_pow()

```
void test1_pow ( )
```

Test Test pow function

Precondition

Long int

Postcondition

Hashed value

5.27 pow_test.h

Go to the documentation of this file.

```
#ifndef _POW_TEST_H
#define _POW_TEST_H

void test1_pow();
void test2_pow();
void test3_pow();

#endif
```

5.28 test.h

```
#ifndef _TEST_H
#define _TEST_H

#pragma GCC diagnostic ignored "-Wpedantic"

#ifndef NOCOLOR
#define KRED  "\x1B[31m"
#define KGRN  "\x1B[32m"
#define KYEL  "\x1B[33m"
#define KCYN  "\x1B[36m"
#define RESET "\033[0m"
#else
#define KRED
#define KGRN
#define KYEL
#define KCYN
#define RESET
#endif

#define PRINT_TEST_RESULT(x) do{                                    \
    __test_counter++;                                               \
    __pass = (x);                                                   \
    __test_passed = (__pass)? __test_passed + 1 : __test_passed;    \
    printf(KYEL "%s" RESET " line "  "%d " KCYN "%s" RESET ": %s\n",    \
       __FILE__, __LINE__ , __FUNCTION__,                           \
       ((!__pass) ? KRED "NOT PASS" RESET : KGRN "PASS" RESET));    \
  } while (0)

#define PRINT_PASSED_PERCENTAGE printf("Tests passed %d%%\n", ((__test_passed * 100) / __test_counter))

static int __test_counter = 0;
static int __test_passed  = 0;
static int __pass = 0;

#endif
```

5.29 test.h

```
#ifndef _TEST_H
#define _TEST_H

#pragma GCC diagnostic ignored "-Wpedantic"

#ifndef NOCOLOR
#define KRED  "\x1B[31m"
#define KGRN  "\x1B[32m"
#define KYEL  "\x1B[33m"
#define KCYN  "\x1B[36m"
#define RESET "\033[0m"
#else
#define KRED
#define KGRN
#define KYEL
#define KCYN
#define RESET
#endif

#define PRINT_TEST_RESULT(x) do{                    \
    __test_counter++;                           \
    __pass = (x);                               \
    __test_passed = (__pass)? __test_passed + 1 : __test_passed;    \
    printf(KYEL "%s" RESET " line "  "%d " KCYN "%s" RESET ": %s\n",    \
       __FILE__, __LINE__ , __FUNCTION__,                \
       ((!__pass) ? KRED "NOT PASS" RESET : KGRN "PASS" RESET));    \
  } while (0)

#define PRINT_PASSED_PERCENTAGE printf("Tests passed %d%%\n", ((__test_passed * 100) / __test_counter))

static int __test_counter = 0;
static int __test_passed  = 0;
static int __pass = 0;

#endif
```

5.30 test.h

```
#ifndef _TEST_H
#define _TEST_H

#pragma GCC diagnostic ignored "-Wpedantic"

#ifndef NOCOLOR
#define KRED  "\x1B[31m"
#define KGRN  "\x1B[32m"
#define KYEL  "\x1B[33m"
#define KCYN  "\x1B[36m"
#define RESET "\033[0m"
#else
#define KRED
#define KGRN
#define KYEL
#define KCYN
#define RESET
#endif

#define PRINT_TEST_RESULT(x) do{                    \
    __test_counter++;                           \
    __pass = (x);                               \
    __test_passed = (__pass)? __test_passed + 1 : __test_passed;    \
    printf(KYEL "%s" RESET " line "  "%d " KCYN "%s" RESET ": %s\n",    \
       __FILE__, __LINE__ , __FUNCTION__,                \
       ((!__pass) ? KRED "NOT PASS" RESET : KGRN "PASS" RESET));    \
  } while (0)

#define PRINT_PASSED_PERCENTAGE printf("Tests passed %d%%\n", ((__test_passed * 100) / __test_counter))

static int __test_counter = 0;
static int __test_passed  = 0;
static int __pass = 0;

#endif
```

5.31 test.h

```
#ifndef _TEST_H
```

```
#define _TEST_H

#pragma GCC diagnostic ignored "-Wpedantic"

#ifndef NOCOLOR
#define KRED "\x1B[31m"
#define KGRN "\x1B[32m"
#define KYEL "\x1B[33m"
#define KCYN "\x1B[36m"
#define RESET "\033[0m"
#else
#define KRED
#define KGRN
#define KYEL
#define KCYN
#define RESET
#endif

#define PRINT_TEST_RESULT(x)                                                    \
  do {                                                                          \
    __test_counter++;                                                           \
    __pass = (x);                                                               \
    __test_passed = (__pass) ? __test_passed + 1 : __test_passed;               \
    printf(KYEL "%s" RESET " line "                                             \
                "%d " KCYN "%s" RESET ": %s\n",                                 \
           __FILE__, __LINE__, __FUNCTION__,                                    \
           ((!__pass) ? KRED "NOT PASS" RESET : KGRN "PASS" RESET));            \
  } while (0)

#define PRINT_PASSED_PERCENTAGE                                                 \
  printf("Tests passed %d%%\n", ((__test_passed * 100) / __test_counter))

static int __test_counter = 0;
static int __test_passed = 0;
static int __pass = 0;

#endif
```

5.32 test.h

```
#ifndef _TEST_H
#define _TEST_H

#pragma GCC diagnostic ignored "-Wpedantic"

#ifndef NOCOLOR
#define KRED  "\x1B[31m"
#define KGRN  "\x1B[32m"
#define KYEL  "\x1B[33m"
#define KCYN  "\x1B[36m"
#define RESET "\033[0m"
#else
#define KRED
#define KGRN
#define KYEL
#define KCYN
#define RESET
#endif

#define PRINT_TEST_RESULT(x) do{                    \
    __test_counter++;                           \
    __pass = (x);                           \
    __test_passed = (__pass)? __test_passed + 1 : __test_passed;    \
    printf(KYEL "%s" RESET " line "  "%d " KCYN "%s" RESET ": %s\n",    \
       __FILE__, __LINE__ , __FUNCTION__,               \
       ((!__pass) ? KRED "NOT PASS" RESET : KGRN "PASS" RESET));    \
  } while (0)

#define PRINT_PASSED_PERCENTAGE printf("Tests passed %d%%\n", ((__test_passed * 100) /
    __test_counter))

static int __test_counter = 0;
static int __test_passed  = 0;
static int __pass = 0;

#endif
```

Index

Arg_hilos, 7
found_value, 7
max, 7
min, 7
target, 8

calcular_solucion
miner.c, 30, 36
miner.h, 43
close_pipes
files.c, 14
files.h, 20
comunicar_logger
miner.c, 30, 36
count_votes
files.c, 15
files.h, 21

die
types.h, 51
die_msg
types.h, 52

ERR
types.h, 52
exit_network
miner.c, 31, 36

f32
types.h, 53
f64
types.h, 53
files.c, 13
close_pipes, 14
count_votes, 15
get_active_pids_unlocked, 15
initialize_mutexes, 15
open_pipes, 16
read_target_unlocked, 16
write_pid_unlocked, 16
write_target_unlocked, 17
write_vote, 17
files.h, 17
close_pipes, 20
count_votes, 21
get_active_pids_unlocked, 21
initialize_mutexes, 21
open_pipes, 22
PID_FILE, 19
PID_MUTEX, 19
READ, 20
read_target_unlocked, 22
TARGET_FILE, 20
TARGET_MUTEX, 20
VOTES_FILE, 20
VOTES_MUTEX, 20
WINNER_MUTEX, 20
WRITE, 20
write_pid_unlocked, 22
write_target_unlocked, 23
write_vote, 23
files_test.c
test1_open_pipes, 56
test2_close_pipes, 56
test3_mutex_init, 56
test4_pids_io, 56
test5_voting_system, 56
files_test.h
test1_open_pipes, 58
test2_close_pipes, 58
test3_mutex_init, 58
test4_pids_io, 58
test5_voting_system, 58
FOUND
types.h, 52
found_value
Arg_hilos, 7

get_active_pids_unlocked
files.c, 15
files.h, 21
get_time_str
miner.c, 31, 36

handler
miner.c, 31, 37

i32
types.h, 53
i64
types.h, 53
id
Logger_args, 8
initialize_mutexes
files.c, 15
files.h, 21

logger
logger.c, 25
logger.h, 27

logger.c, 24
logger, 25
logger.h, 26
logger, 27
Logger_args, 8
id, 8
pos_votes, 8
solution, 9
target, 9
validated, 9
votes, 9
wallets, 9
winner, 9

main
pow_test.c, 65
max
Arg_hilos, 7
MAX_MINERS
types.h, 52
MAX_TRIES
types.h, 52
MIN
types.h, 52
min
Arg_hilos, 7
MIN_MINERS
types.h, 53
miner.c, 28
calcular_solucion, 30, 36
comunicar_logger, 30, 36
exit_network, 31, 36
get_time_str, 31, 36
handler, 31, 37
miner_set_alarm, 31, 37
minero, 32, 37
pow_seek, 32, 38
start_mining, 33, 39
start_voting, 33, 39
TARGET_INIT, 30, 35
timeout, 33, 39
wait_more_miners, 32, 38
wait_votes, 33, 38
miner.h, 39
calcular_solucion, 43
minero, 41, 44
wait_more_miners, 44
wait_votes, 44
Miner_data, 10
n_threads, 10
time, 10
Miner_Mutexes, 10
pid, 11
tgt, 11
vot, 11
win, 11
miner_set_alarm
miner.c, 31, 37
miner_test.c
start_voting, 61
test1_calcular_solucion, 60
test1_open_pipes, 60
test1_wait_more_miners_success, 60
test1_wait_votes_success, 61
test2_wait_more_miners_timeout, 61
test2_wait_votes_timeout, 61
timeout, 61
miner_test.h
test1_calcular_solucion, 63
test1_wait_more_miners_success, 63
test1_wait_votes_success, 63
test2_wait_more_miners_timeout, 63
test2_wait_votes_timeout, 63
minero
miner.c, 32, 37
miner.h, 41, 44
mrush.c, 45
str_to_u64, 46

n_threads
Miner_data, 10

OK
types.h, 53
open_pipes
files.c, 16
files.h, 22

pid
Miner_Mutexes, 11
PID_FILE
files.h, 19
PID_MUTEX
files.h, 19
pos_votes
Logger_args, 8
pow.h, 47
pow_hash, 48
POW_LIMIT, 48
pow_hash
pow.h, 48
POW_LIMIT
pow.h, 48
pow_seek
miner.c, 32, 38
pow_test.c
main, 65
test1_pow, 65
pow_test.h
test1_pow, 67

READ
files.h, 20
read_target_unlocked
files.c, 16
files.h, 22

solution

Logger_args, 9
start_mining
miner.c, 33, 39
start_voting
miner.c, 33, 39
miner_test.c, 61
str_to_u64
mrush.c, 46

target
Arg_hilos, 8
Logger_args, 9
TARGET_FILE
files.h, 20
TARGET_INIT
miner.c, 30, 35
TARGET_MUTEX
files.h, 20
Test List, 1
test1_calcular_solucion
miner_test.c, 60
miner_test.h, 63
test1_open_pipes
files_test.c, 56
files_test.h, 58
miner_test.c, 60
test1_pow
pow_test.c, 65
pow_test.h, 67
test1_wait_more_miners_success
miner_test.c, 60
miner_test.h, 63
test1_wait_votes_success
miner_test.c, 61
miner_test.h, 63
test2_close_pipes
files_test.c, 56
files_test.h, 58
test2_wait_more_miners_timeout
miner_test.c, 61
miner_test.h, 63
test2_wait_votes_timeout
miner_test.c, 61
miner_test.h, 63
test3_mutex_init
files_test.c, 56
files_test.h, 58
test4_pids_io
files_test.c, 56
files_test.h, 58
test5_voting_system
files_test.c, 56
files_test.h, 58
tgt
Miner_Mutexes, 11
time
Miner_data, 10
timeout
miner.c, 33, 39
miner_test.c, 61
types.h, 49
die, 51
die_msg, 52
ERR, 52
f32, 53
f64, 53
FOUND, 52
i32, 53
i64, 53
MAX_MINERS, 52
MAX_TRIES, 52
MIN, 52
MIN_MINERS, 53
OK, 53
u32, 53
u64, 54

u32
types.h, 53
u64
types.h, 54
unit_tests/files_test/files_test.c, 54
unit_tests/files_test/files_test.h, 57, 59
unit_tests/files_test/test.h, 67
unit_tests/logger_test/logger_test.h, 59
unit_tests/logger_test/test.h, 68
unit_tests/miner_test/miner.c, 34
unit_tests/miner_test/miner.h, 41, 45
unit_tests/miner_test/miner_test.c, 59
unit_tests/miner_test/miner_test.h, 62, 64
unit_tests/miner_test/test.h, 68
unit_tests/pow_test/pow_test.c, 64
unit_tests/pow_test/pow_test.h, 66, 67
unit_tests/pow_test/test.h, 68
unit_tests/test.h, 69

validated
Logger_args, 9
vot
Miner_Mutexes, 11
votes
Logger_args, 9
VOTES_FILE
files.h, 20
VOTES_MUTEX
files.h, 20

wait_more_miners
miner.c, 32, 38
miner.h, 44
wait_votes
miner.c, 33, 38
miner.h, 44
wallets
Logger_args, 9
win
Miner_Mutexes, 11
winner

Logger_args, 9
WINNER_MUTEX
files.h, 20
WRITE
files.h, 20
write_pid_unlocked
files.c, 16
files.h, 22
write_target_unlocked
files.c, 17
files.h, 23
write_vote
files.c, 17
files.h, 23